

AIDA 실험 결과 설명자료

비전공자 팀원 공유용 | 테스트 결과, 차트 해석, 프로그램 작동 흐름

한 줄 요약: AIDA는 AI 학습 데이터의 라벨 품질을 검사하고, 어떤 라벨 오류를 먼저 재검수해야 하는지 알려주는 데이터 품질 점검 프로그램입니다.

1. 이번 테스트가 확인한 것

자동차 이미지 라벨에 가로 길이 오류, 세로 길이 오류, 회전각 오류를 일부러 넣은 뒤 YOLOv8n 모델을 학습시켰습니다. 정상 라벨로 학습한 기준선과 비교해 오류 종류별로 AI 성능이 얼마나 떨어지는지 확인했습니다.

항목	내용
데이터	KITTI Car 클래스, 학습 400장 / 평가 120장
모델	YOLOv8n, COCO 사전학습 가중치에서 파인튜닝
오류 조건	정상 1개 + 가로 4개 + 세로 4개 + 회전 4개 = 총 13개
오류 비율	전체 라벨 중 30%에만 오류 주입
학습 기준	13개 조건 모두 동일하게 50epoch
실행 환경	Windows, NVIDIA RTX 3050 CUDA
최종 결과 파일	backend/app/data/metrics.csv

2. 핵심 결과

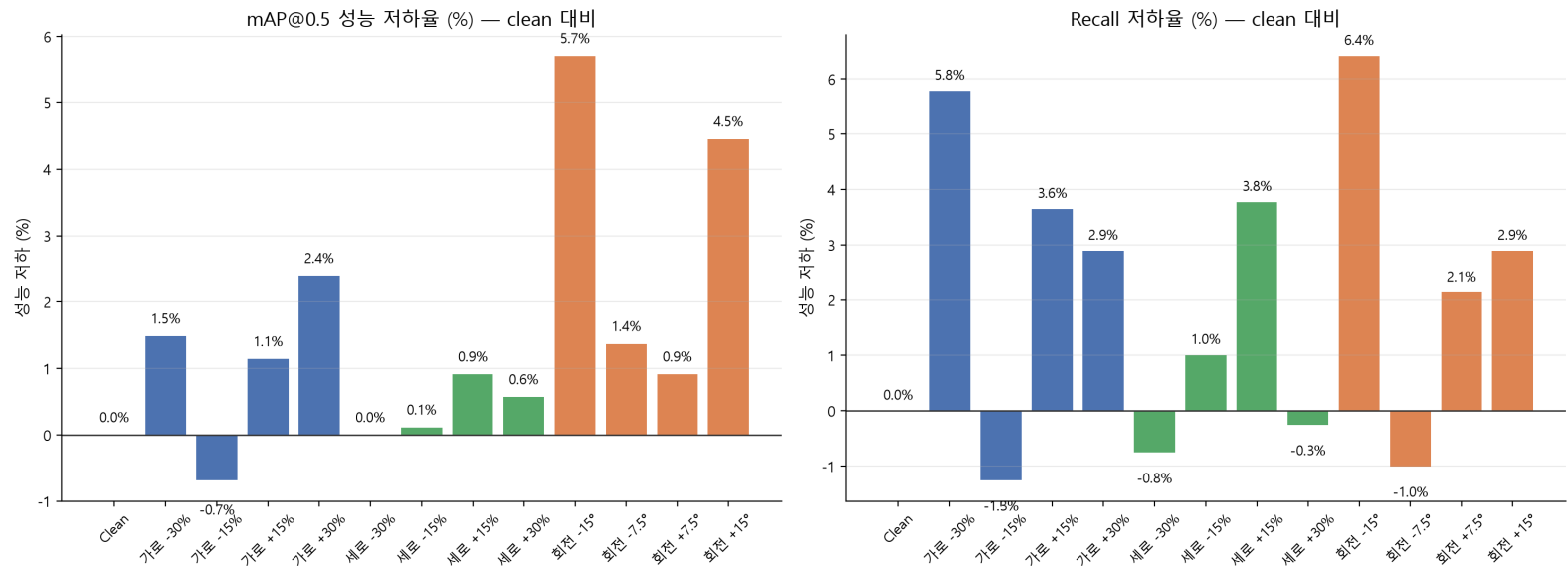
정상 기준 데이터(clean)의 mAP@0.5는 0.876, Precision은 0.859, Recall은 0.795입니다. 이 값을 기준으로 각 오류 조건의 성능 하락률을 계산했습니다.

순위	조건	오류 의미	mAP@0.5	성능 변화
1	rot_m15	회전각 오류 -15도	0.826	-5.7%
2	rot_p15	회전각 오류 +15도	0.837	-4.5%
3	width_p30	가로 길이 오류 +30%	0.855	-2.4%
4	width_m30	가로 길이 오류 -30%	0.863	-1.5%

해석: 이번 실험에서는 회전각 오류가 가로/세로 크기 오류보다 성능을 더 크게 떨어뜨리는 경향을 보였습니다. 특히 -15도 회전 조건(rot_m15)이 가장 큰 하락을 보였습니다.

3. 차트로 보는 결과

오류 유형별 성능 저하 패턴 (13개 조건 전체, KITTI Car 520장, YOLOv8n, 동일 50epoch)



13개 조건 전체를 동일 50epoch로 재학습(CUDA) — epoch 불일치 한계 해소. 단일 시드 실행이라 소규모 변동은 노이즈 가능

그림 1. 13개 조건 전체의 mAP@0.5 성능 저하율. 막대가 클수록 정상 기준 대비 성능이 더 많이 떨어진 조건입니다.

4. 결과를 쉽게 말하면

결론	쉬운 설명
회전 오류가 가장 위험	자동차를 감싸는 네모 박스가 기울어지면 AI가 위치와 크기를 더 헛갈려 합니다.
가로/세로 오류는 영향이 비교적 작음	박스가 조금 크거나 작은 정도는 모델이 어느 정도 버틸 수 있었습니다.
작은 오류는 수치가 흔들릴 수 있음	조건당 1회 실행이라 작은 차이는 학습 과정의 우연한 흔들림이 섞였을 수 있습니다.
AIDA의 역할	오류를 100% 확정하는 도구가 아니라, 재검수 우선순위를 잡아주는 가이드입니다.

5. 조건별 상세 결과

아래 표는 CSV에 들어 있는 주요 수치를 문서용으로 정리한 것입니다. 별도로 전달할 metrics.csv에는 같은 결과가 원본 데이터 형태로 들어 있습니다.

조건	유형	강도	mAP@0.5	mAP@0.5:0.95	Precision	Recall	성능 변화
clean	기준선	0	0.876	0.626	0.859	0.795	-0.0%
width_m30	가로 길이 오류	-30%	0.863	0.584	0.882	0.749	-1.5%
width_m15	가로 길이 오류	-15%	0.882	0.602	0.838	0.805	+0.7%
width_p15	가로 길이 오류	+15%	0.866	0.590	0.865	0.766	-1.1%
width_p30	가로 길이 오류	+30%	0.855	0.564	0.848	0.772	-2.4%
height_m30	세로 길이 오류	-30%	0.876	0.587	0.872	0.801	-0.0%
height_m15	세로 길이 오류	-15%	0.875	0.594	0.858	0.787	-0.1%
height_p15	세로 길이 오류	+15%	0.868	0.597	0.884	0.765	-0.9%
height_p30	세로 길이 오류	+30%	0.871	0.582	0.852	0.797	-0.6%
rot_m15	회전각 오류	-15도	0.826	0.554	0.840	0.744	-5.7%
rot_m7_5	회전각 오류	-7.5도	0.864	0.570	0.845	0.803	-1.4%
rot_p7_5	회전각 오류	+7.5도	0.868	0.579	0.851	0.778	-0.9%
rot_p15	회전각 오류	+15도	0.837	0.555	0.811	0.772	-4.5%

6. 프로그램 로직은 어떻게 진행되는가

단계	폴더/파일	하는 일	비유
1	experiment/download_kitti.py	자동차 이미지와 라벨 데이터를 준비	시험지를 가져오는 단계
2	experiment/data_loader.py	실험에 필요한 자동차 데이터만 정리	시험 문제를 정리하는 단계
3	experiment/error_injector.py	정상 라벨 일부에 가로/세로/회전 오류를 주입	일부러 틀린 정답지를 만드는 단계
4	experiment/train.py	오류 조건별로 YOLO 모델을 학습	AI가 각 정답지로 공부하는 단계
5	experiment/evaluate.py	평가 이미지에서 성능을 측정	AI가 시험을 보는 단계
6	experiment/run_all.py	13개 조건을 순서대로 실행하고 CSV 저장	전체 시험 결과표를 만드는 단계
7	backend API	metrics.csv를 읽어 요약/조건/진단 데이터 제공	결과표를 화면용으로 번역
8	frontend Dashboard	그래프, 상세표, 재검수 우선순위 표시	팀원이 보는 최종 화면

7. 대시보드가 보여주는 것

화면 요소	의미
데이터셋 품질 점수	현재 샘플 데이터의 품질을 100점 기준으로 표시
오류 조건별 성능 저하 그래프	어떤 오류 조건에서 AI 성능이 얼마나 떨어졌는지 표시
조건별 상세 지표 표	mAP, Precision, Recall 등 수치를 조건별로 표시
CSV 다운로드	조건별 결과를 파일로 내려받아 따로 공유 가능
재검수 우선순위 표	어떤 라벨 오류를 먼저 확인해야 하는지 안내

8. 용어정리

용어	쉬운 뜻
라벨	이미지 속 자동차 위치를 표시한 정답 정보
바운딩박스	자동차를 감싸는 네모 박스
객체탐지	이미지에서 자동차 같은 물체를 찾아 위치까지 표시하는 AI 기술
YOLOv8n	빠르고 가벼운 객체탐지 AI 모델
mAP@0.5	AI가 물체를 얼마나 잘 찾았는지 보는 대표 점수
Precision	AI가 자동차라고 예측한 것 중 실제 자동차가 얼마나 많았는지
Recall	실제 자동차 중 AI가 놓치지 않고 찾은 비율
epoch	같은 학습 데이터를 몇 번 반복해서 공부했는지
clean	오류를 넣지 않은 정상 기준 데이터
재검수 우선순위	사람이 라벨을 다시 확인할 때 어디부터 볼지 정한 순서

9. 최종 요약

- 테스트는 완료되었고 결과는 backend/app/data/metrics.csv에 반영되어 있습니다.
- 정상 기준 성능은 mAP@0.5 기준 0.876입니다.
- 가장 큰 성능 저하는 rot_m15 조건에서 나타났고, 정상 대비 5.7% 하락했습니다.
- AIDA는 오류를 100% 확정하는 도구가 아니라, 재검수 우선순위를 잡아주는 도구입니다.
- 현재 결과는 조건당 1회 실행이므로, 후속 단계에서는 반복 실험으로 신뢰도를 높이면 좋습니다.